

DAY 39 – 46 · BEGINNER-FRIENDLY PRACTICE NOTES

GitHub Actions & CI/CD

From CI/CD concepts to workflows, runners, jobs, secrets, artifacts, Docker publishing & reusable automation

39 · CI/CD Concepts

40 · First Workflow

41 · Triggers & Matrix

42 · Runners

43 · Jobs & Env

44 · Secrets & Artifacts

45 · Docker Publish

46 · Reusable Workflows

Read the concept → type the example → push to GitHub → inspect the run → break it → fix it → write what you learned

DAY 39 What is CI/CD?

Why CI/CD exists

CI vs Delivery vs Deployment

Pipeline anatomy

Reading a workflow

Beginner idea: without automation, developers manually build, test, deploy — risky as a team grows. CI/CD is a **practice**, not one tool (GitHub Actions, Jenkins, GitLab CI, CircleCI all implement it). A failed pipeline is useful — it catches a problem early.

CI vs CD

Term	Meaning
Continuous Integration	Developers frequently integrate changes; automated tests/builds catch problems early.
Continuous Delivery	Code is automatically prepared and kept release-ready, but production release can still need approval.
Continuous Deployment	A successful pipeline automatically deploys changes to production.

Pipeline Anatomy

Trigger → Stage → Job → Step → Runner → Artifact

Trigger starts the pipeline (push/PR) · **Stage** = logical phase (build/test/deploy) · **Job** = unit of work in a stage · **Step** = one command/action · **Runner** = machine executing the job · **Artifact** = output like a report or build file.

PRACTICE

- Think about 5 developers manually deploying — list what can go wrong; explain "it works on my machine."
- Draw: Developer push → Test → Docker Build → Staging Deploy.
- Explore an open-source repo's [.github/workflows/](#) — note trigger, job count, purpose.

EXPECTED OUTPUT

day-39-cicd-concepts.md · a pipeline diagram (hand-drawn or text-based)

DAY 40 Your First GitHub Actions Workflow

on: jobs: runs-on: steps: uses: run:

Setup

Create a public repo named `github-actions-practice` → clone it locally → create `.github/workflows/`.

Starter Workflow — hello.yml

Triggers on every push · one job named `greet` · runs on `ubuntu-latest` · checks out code · prints a greeting.

Practice YAML — extra steps

```
- name: Show branch
  run: echo "Branch: ${github.ref_name}"
- name: List files
  run: ls -la
- name: Runner OS
  run: uname -a
- name: Current date
  run: date
```

Break it on purpose: add a step that fails, e.g. `exit 1`. Push it, inspect the red run, fix it, push again.

EXTRA STEPS TO ADD

- Print current date/time
- Print the triggering branch via GitHub's branch context
- List repository files
- Print the runner operating system

EXPECTED OUTPUT

hello.yml · day-40-first-workflow.md · screenshot of a green pipeline run

DAY 41 Triggers & Matrix Builds

Trigger	Runs when...
push	Code is pushed
pull_request	PR activity happens
schedule	Using a cron expression
workflow_dispatch	Run manually, optionally with inputs

PR Practice

`pr-check.yml` — trigger only when a PR opens/updates against `main`; print the branch name.

Schedule Practice

Daily at midnight UTC: `0 0 * * *`. Every Monday 9AM UTC: `0 9 * * 1`.

Manual Workflow

`manual.yml` with `workflow_dispatch` and an environment input (staging/production) — print the selected value.

Matrix Builds

A matrix runs the same job for multiple combinations. 3 Python versions × 2 OS values = **6 combinations** before exclusions.

exclude: removes one combo **fail-fast: true (default)** cancels other jobs on failure **fail-fast: false** lets others continue

PRACTICE

Watch matrix jobs run in parallel → add an exclusion → deliberately fail one job → compare `fail-fast: true` vs `false`.

EXPECTED OUTPUT

pr-check.yml, manual.yml, matrix.yml (and/or updated workflows) · day-41-triggers.md · run screenshots

DAY 41 Useful YAML Patterns

```
# Pull request
on:
  pull_request:
    branches: [main]

# Schedule
on:
  schedule:
    - cron: '0 0 * * *'

# Manual
on:
  workflow_dispatch:
    inputs:
      environment:
        description: Environment
        required: true

# Matrix
strategy:
  fail-fast: false
  matrix:
    python-version: ['3.10', '3.11', '3.12']
    os: [ubuntu-latest, windows-latest]
    exclude:
      - os: windows-latest
        python-version: '3.10'
```

DAY 42 Runners: GitHub-Hosted & Self-Hosted

Every job needs a machine — a **runner** executes the job. **GitHub-hosted**: GitHub provides/manages it. **Self-hosted**: you provide and manage the machine (local or cloud VM).

GitHub-Hosted Practice

[ubuntu-latest](#) [windows-latest](#) [macos-latest](#)

3 jobs, one per OS — each prints OS name, hostname, current user. On [ubuntu-latest](#) also print Docker, Python, Node and Git versions — knowing what's pre-installed simplifies setup.

Self-Hosted Runner Setup

- Repository → Settings → Actions → Runners → New self-hosted runner → choose Linux → follow GitHub's generated instructions
- Start the runner and verify it shows **Idle** with a green dot
- Create [self-hosted.yml](#) with [runs-on: self-hosted](#) — print hostname and working directory, create a file, verify it exists on the machine after the job
- Add a label like [my-linux-runner](#) and target with [runs-on: \[self-hosted, my-linux-runner\]](#)

Comparison

	GitHub-Hosted	Self-Hosted
Who manages it?	GitHub	You / your organization
Cost	Uses GitHub-hosted minutes/plan limits	Your machine/cloud infra cost
Pre-installed tools	Runner image provides tools	You control/install tools
Good for	General CI/CD	Special hardware, private network, custom environments
Security concern	Less machine management for you	You must secure and maintain the runner

DAY 43 Jobs, Steps, Env Vars & Conditionals

Multi-Job Dependency Chain

build → **needs** → **test** → **needs** → **deploy**

Environment Variables — 3 Scopes

Scope	Example
Workflow-level	<code>APP_NAME: myapp</code>
Job-level	<code>ENVIRONMENT: staging</code>
Step-level	<code>VERSION: 1.0.0</code>

Print them in one step, plus the commit SHA and actor via GitHub context.

Job Outputs

One job creates an output (e.g. today's date); another reads it with `needs.<job>.outputs.<name>`.

```
echo "date=$(date)" >> $GITHUB_OUTPUT
```

Conditionals

- Run a step only on `main`
- Run a step only when the previous step failed
- Run a job only on push events, not pull requests
- `continue-on-error: true` lets a step fail without failing the whole job

Smart pipeline: trigger on push to any branch → run lint and test in parallel → then a summary job after both, identifying main vs feature branch and printing the commit message.

PRACTICE

Make jobs independent first → then add `needs` → add variables at each scope → pass an output between jobs → deliberately fail a step and test the failure condition.

EXPECTED OUTPUT

multi-job.yml, smart-pipeline.yml and other practice workflows · day-43-jobs-steps.md

DAY 43 Useful YAML Patterns

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Building the app"
  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Running tests"
  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying"

# Job output idea
# echo "date=$(date)" >> $GITHUB_OUTPUT
# Read with:
# ${ needs.job-name.outputs.date }
```

DAY 44 Secrets, Artifacts & Real Tests in CI

Secrets

- Create repo secret `MY_SECRET_MESSAGE` in Settings → Secrets and Variables → Actions
- Use the secret without hardcoding it — never print its actual value in logs
- GitHub auto-masks secret values in logs, but design workflows so secrets stay unexposed
- Prepare `DOCKER_USERNAME` and `DOCKER_TOKEN` secrets for Day 45

Artifacts

An artifact is a file saved from a run (test report, log). Upload with `actions/upload-artifact@v4`, retrieve in another job with `actions/download-artifact@v4`.

Real Tests & Caching

Put a Python/Shell script in the repo → checkout → install deps → run script → non-zero exit fails the pipeline. Break it intentionally, see red, fix it, see green. Use `actions/cache@v4` for dependencies — run twice and compare time.

PRACTICE

Create a test-report.txt/log and upload it → download it from the Actions tab → pass an artifact between two jobs → add a real script and test both success and failure.

EXPECTED OUTPUT

day-44-secrets-artifacts.md · artifact download screenshot · passing CI test screenshot

DAY 44 Secrets, Artifacts & Cache — Quick Patterns

```
# Secret as environment variable
env:
  MY_MESSAGE: ${ secrets.MY_SECRET_MESSAGE }
run: |
  if [ -n "$MY_MESSAGE" ]; then
    echo "The secret is set: true"
  fi

# Upload
- uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: test-report.txt

# Download in another job
- uses: actions/download-artifact@v4
  with:
    name: test-report
```

DAY 45 Docker Build & Push in GitHub Actions

A push to GitHub automatically builds a Docker image and publishes it to Docker Hub — connecting source control, CI and container distribution.

Pipeline Journey

git push → GA trigger → checkout → Docker build → tag → Docker Hub login → push → pull → run container

Prepare

Use the app from Day 36 or another simple Dockerfile — add it to `github-actions-practice`. Ensure `DOCKER_USERNAME` and `DOCKER_TOKEN` secrets exist.

Tags & Branch Rule

- Publish a `latest` tag and a commit-based tag like `sha-<short-commit-hash>` — makes a specific build easy to identify
- Feature branches can build the image, but the **push step should only run for main** — validate builds without publishing every branch's image

`docker/login-action@v3`

`docker/build-push-action@v5`

PRACTICE

Build on push to main → verify logs → log in with secrets → push both tags → push a feature branch and verify it builds but does not publish → pull and run the image locally or on a cloud server.

EXPECTED OUTPUT

docker-publish.yml · Docker Hub image · README status badge · day-45-docker-cicd.md + pipeline screenshot

DAY 45 Docker Publishing Workflow — Simplified

```
name: Docker Publish
on:
  push:
    branches: [main]
jobs:
  docker:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_TOKEN }
      - uses: docker/build-push-action@v5
        with:
          push: true
          tags: yourusername/yourrepo:latest
```

DAY 46 Reusable Workflows & Composite Actions

Real teams avoid copying the same workflow logic into every repo. Reusable workflows and composite actions help standardize automation.

Reusable Workflow

- Designed to be called by another workflow — trigger is `workflow_call`
- Lives under `.github/workflows/`
- A caller invokes it with a job-level `uses` reference (unlike a normal action used inside a step)

Build It

`reusable-build.yml` with `workflow_call`. Inputs: `app_name` (string, required), `environment` (string, required, default staging). Secret: `docker_token` (required). Job checks out code, prints app/environment, confirms the token is set without printing it.

Caller & Outputs

`call-build.yml` triggers on push to main and calls `./.github/workflows/reusable-build.yml` with inputs + Docker token secret. Expose `build_version` (e.g. `v1.0-<short-sha>`) — a second caller job waits with `needs` and prints the returned version.

Composite Action

`.github/actions/setup-and-greet/action.yml` — inputs `name` and `language` (default en). Uses `runs: using: "composite"`; prints greeting, date, runner OS, and output `greeted=true`. Called with `uses: ./.github/actions/setup-and-greet`.

Comparison

	Reusable Workflow	Composite Action
Triggered by	<code>workflow_call</code>	<code>uses:</code> in a step
Can contain jobs?	Yes	No — used within a job's step sequence
Multiple steps?	Yes	Yes
Lives where?	<code>.github/workflows/</code>	<code>.github/actions/<action>/action.yml</code>
Accepts secrets directly?	Yes, via <code>workflow_call</code> secrets	Pass as inputs/env instead
Best for	Reusable multi-job pipeline logic	Reusable step-level logic

DAY 46 Reusable Workflow & Composite Action — Structure

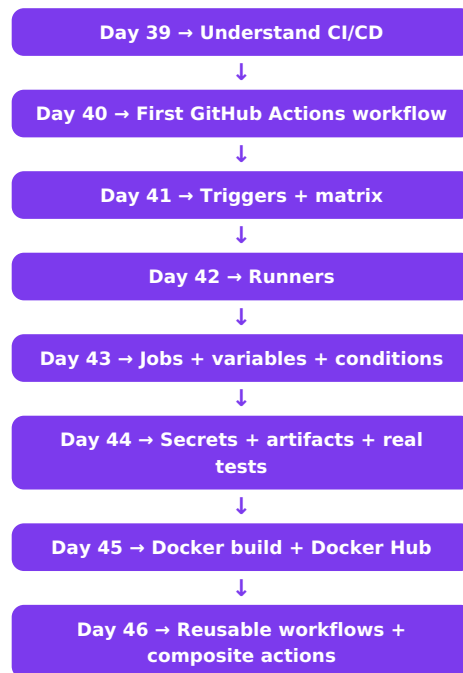
```
# .github/workflows/reusable-build.yml
on:
  workflow_call:
    inputs:
      app_name:
        required: true
        type: string
      environment:
        required: true
        type: string
        default: staging
    secrets:
      docker_token:
        required: true
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Building ${ inputs.app_name } for ${ inputs.environment }"
```

```
# .github/workflows/call-build.yml
on:
  push:
    branches: [main]
jobs:
  build:
    uses: ../.github/workflows/reusable-build.yml
    with:
      app_name: "my-web-app"
      environment: "production"
    secrets:
      docker_token: ${ secrets.DOCKER_TOKEN }
```

```
# .github/actions/setup-and-greet/action.yml
name: Setup and Greet
inputs:
  name:
    required: true
  language:
    default: en
runs:
  using: composite
  steps:
    - shell: bash
      run: echo "Hello ${ inputs.name }"
```

FINAL Revision — Days 39 to 46

Think of the eight days as one journey:



FINAL Beginner Self-Check

- ☐ Can I explain CI, Continuous Delivery and Continuous Deployment in my own words?
- ☐ Can I find a workflow under `.github/workflows/`?
- ☐ Do I understand trigger, job, step and runner?
- ☐ Can I create a workflow that runs on push?
- ☐ Can I use a pull request, schedule and manual trigger?
- ☐ Can I explain why a matrix creates multiple jobs?
- ☐ Can I tell GitHub-hosted and self-hosted runners apart?
- ☐ Can I use `needs` to create a job dependency?
- ☐ Can I use environment variables and GitHub context values?
- ☐ Can I pass an output from one job to another?
- ☐ Can I store and use secrets without printing them?
- ☐ Can I upload/download an artifact?
- ☐ Can I make a failing test turn the pipeline red?
- ☐ Can I build and publish a Docker image from GitHub Actions?
- ☐ Can I explain reusable workflow vs composite action?

PRACTICE RULE

Don't only copy YAML — type it, run it, inspect every step, break it deliberately, read the error, fix it, and write one sentence explaining why it worked.

Suggested Repository Structure

<code>.github/workflows/hello.yml</code>	Day 40 starter workflow
<code>.github/workflows/pr-check.yml</code>	Day 41 PR trigger
<code>.github/workflows/manual.yml</code>	Day 41 workflow_dispatch
<code>.github/workflows/matrix.yml</code>	Day 41 matrix build
<code>.github/workflows/self-hosted.yml</code>	Day 42 self-hosted runner
<code>.github/workflows/multi-job.yml</code>	Day 43 job dependencies
<code>.github/workflows/smart-pipeline.yml</code>	Day 43 lint + test + summary
<code>.github/workflows/docker-publish.yml</code>	Day 45 Docker build & push
<code>.github/workflows/reusable-build.yml</code>	Day 46 reusable workflow
<code>.github/workflows/call-build.yml</code>	Day 46 caller workflow
<code>.github/actions/setup-and-greet/action.yml</code>	Day 46 composite action
<code>Dockerfile</code> , <code>README.md</code>	App + docs